You have no setup in this test (in part because you're not requiring any security or login at this point). The test uses Capybara methods to interact with the application to simulate user interaction.

Prescription 7	Using Capybara allows you to simulate user activity for end-to-end tests of your Rails features.

Start by using the Capybara method visit to simulate a request to your application at the URL that matches the route new_project_path. Once it gets to that page, it uses the Capybara method fill_in to put text in a couple of form fields, then it clicks a button labeled Create Project using the click_on method. I'll talk in more detail about the Capybara API in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151. Right now it's enough to get the gist of what the test is doing.

Finally, on line 10, you enter the evaluation phase of the test by visiting a route, projects_path, that represents the project index page and asserting that the title of the new task appears on the page, as does the total size of the project. A task of size 3 points and a task of size 5 points means you're looking for a total of 8 points.

You're not making any assumptions about the layout or presentation of the page—only that the new task name is there. Typically, when doing an end-to-end test the goal is to have the success criteria be based on something that is visible in a response rather than checking the database to see if the object is created. Later I'll talk about ways to incorporate assumptions about the layout of the page to help make the test more descriptive of the expected output.

This is a reasonable end-to-end test. It simulates a simple workflow by filling out a form, submitting it, and validating at least part of the resulting data.

There are several reasons why it's valuable to have a test like this one that works from outside the application:

- It makes no assumptions about the structure of the underlying code.
- It forces you to think of your feature in terms of behavior that is visible to a user or client of the application. Not all features have user-facing components, but where they do, being able to specify correct behavior without regard to the implementation is valuable.